

How well the agents perform, measured

Every agent run through the live analysis endpoint on all 987 real sprints from the TAWOS dataset, plus the published evaluations re-run and every main feature timed. No language-model key was used, which is how the product ships. Each chart says what it shows, what it lets you claim, and what it does not.

Data: 31 open-source projects, 22 of them held out and never used for tuning

Measured 21–22 September 2026 Source: VALIDATION_REPORT.md

Real-sprint analyses

2,961

0 failed checks across 29 kinds of check

Warning halfway through a sprint

AUC 0.792

on 22 projects it never saw; F1 0.706

Fix predictions that held

1,524 / 1,524

severity dropped a level in 62%

Prompt saving at 2,003 tasks

487×

1.47× growth while the project grows 154×

1 · CORRECTNESS

Does each agent compute exactly what it claims?

Every check below ran on every real-sprint analysis. The Planning, Progress and Workload figures were compared with an independent recount from the raw sprint, not with the agents' own code.

Coordinator agent

Returns all five specialists 1,974 / 1,974 pass

Analysis completes 1,974 / 1,974 pass

Same sprint, same answer 987 / 987 pass

Planning agent

Figures match an independent recount 1,974 / 1,974 pass

No error output 1,974 / 1,974 pass

Output valid 1,974 / 1,974 pass

Progress agent

Completion rate matches recount	1,974 / 1,974	pass
No error output	1,974 / 1,974	pass
Output valid	1,974 / 1,974	pass
Stalled tasks match recount	1,974 / 1,974	pass

Workload agent

Team, overloaded and under-used match recount	1,974 / 1,974	pass
No error output	1,974 / 1,974	pass
Output valid	1,974 / 1,974	pass

Risk agent

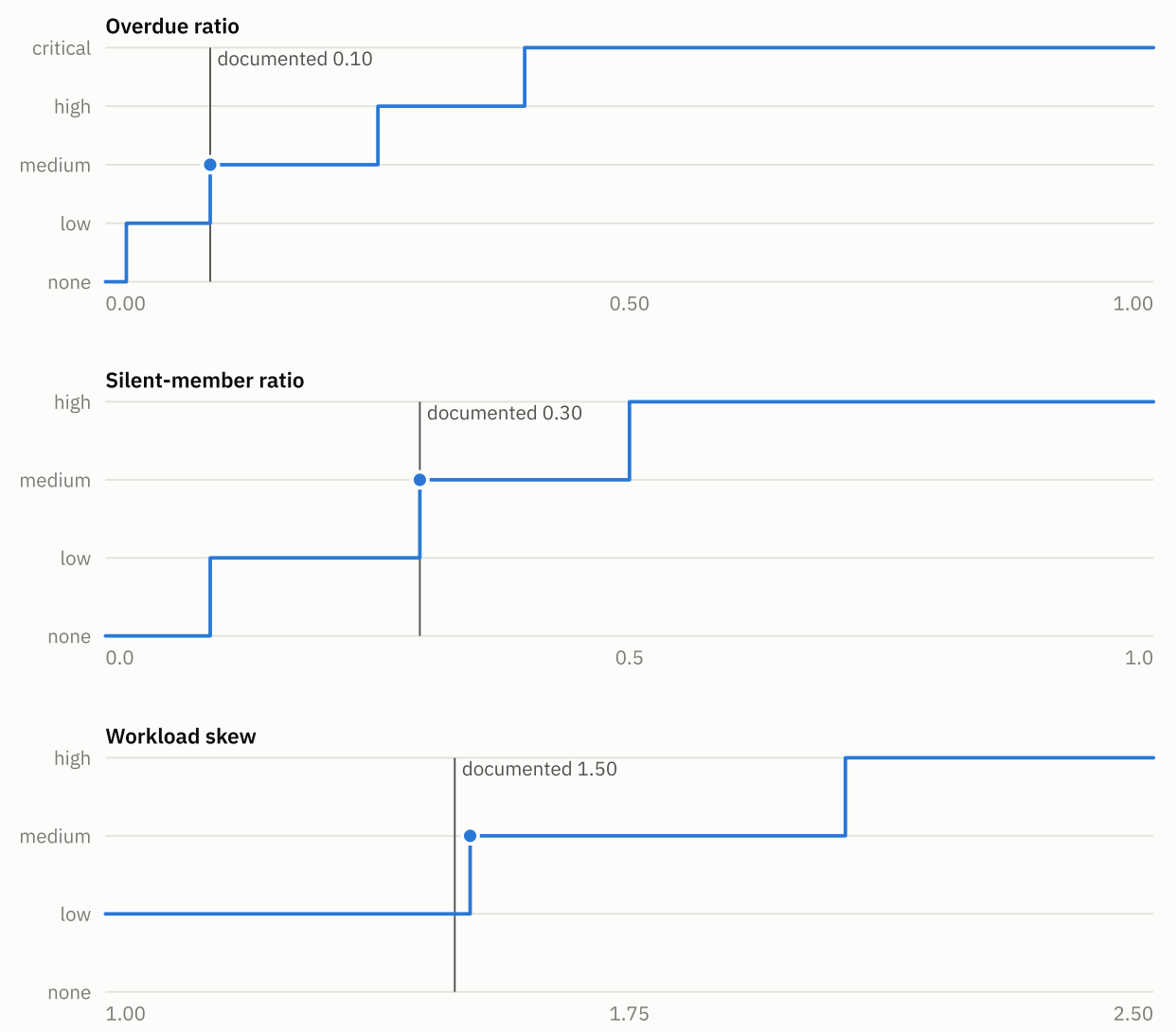
Every evidence reference resolves	7,831 / 7,831	pass
Every cited node is real	40,001 / 40,001	pass
Every finding cites evidence	7,102 / 7,102	pass
Scores match a direct graph computation	1,974 / 1,974	pass
No error output	1,974 / 1,974	pass
Output valid	1,974 / 1,974	pass

Recommendation agent

Starting load matches the board	1,524 / 1,524	pass
Effect stated with every move	1,524 / 1,524	pass
Every move is valid	1,524 / 1,524	pass
No move raises the heaviest load	1,524 / 1,524	pass
Predicted heaviest load = actual	1,524 / 1,524	pass
Predicted severity = actual	1,524 / 1,524	pass
Covers every medium+ risk	4,363 / 4,363	pass
Grounded in an actual finding	4,363 / 4,363	pass
No error output	1,974 / 1,974	pass
Output valid	1,974 / 1,974	pass

Each detector changes its verdict exactly at its documented threshold

Severity as the measured signal is swept across its range on generated projects. The vertical rule marks the threshold the documentation states.



Signal	Documented threshold	Last reading below	First reading above
Overdue ratio	0.10	0.08 (low)	0.10 (medium)
Silent-member ratio	0.30	0.2 (low)	0.3 (medium)
Workload skew	1.50	1.43 (low)	1.52 (medium)

WHAT IT SHOWS

Across **2,961 analyses** nothing failed: every agent completed, every output matched its schema, the same sprint always produced the same answer, and all **40,001** tasks and people cited as evidence exist in that sprint's graph. On 180 generated projects with a planted problem, detection is exact for all six risk types (F1 1.00), and each detector switches at its documented value.

YOU CAN CLAIM

The agents are **correct and deterministic**: they report the true figures for the data they are given, their evidence is never invented, and their thresholds behave as documented.

DON'T CLAIM

That this makes the verdicts *useful*. Correct computation of a signal is not the same as that signal predicting anything — section 2 measures that separately.

2 · PREDICTION ON UNSEEN PROJECTS

Does it warn about sprints that really finish late?

270 sprints from 22 projects the system never saw. A sprint counts as late when 30% or more of its issues were still open at its end. Every predictor is scored against the same sprints.

F1 score by when the verdict is made

Held-out sprints. Higher is better; 1.0 is perfect.

TeamSync risk agent Tuned logistic model Flag every sprint

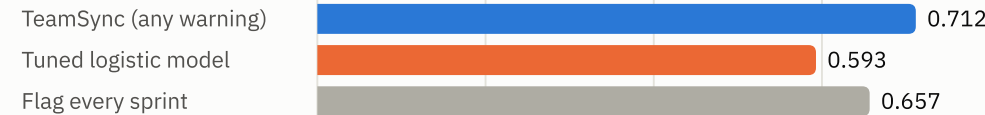
Halfway



Three-quarters



Sprint end

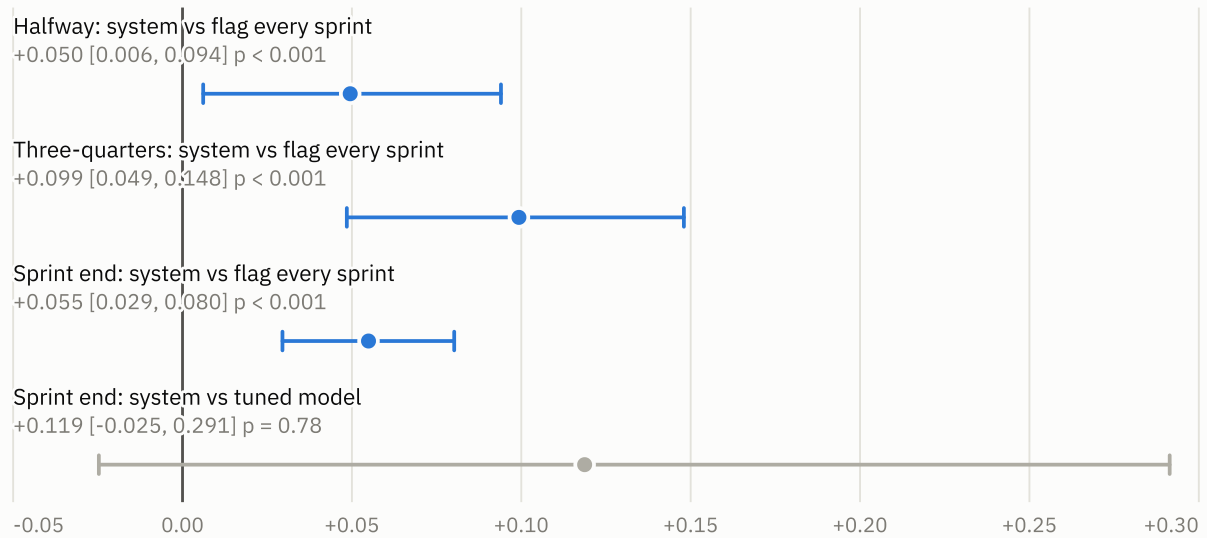


When	Predictor	F1
Halfway	TeamSync (any warning)	0.706
Halfway	Flag every sprint	0.657
Three-quarters	TeamSync (any warning)	0.756
Three-quarters	TeamSync (high+ only)	0.803
Three-quarters	Flag every sprint	0.657
Sprint end	TeamSync (any warning)	0.712
Sprint end	Tuned logistic model	0.593
Sprint end	Flag every sprint	0.657

Is the difference real? Paired F1 gain with 95% interval

Same sprints, resampled by project 2,000 times. An interval that crosses zero is not a real difference.

■ Real difference ■ Cannot be separated



Comparison	F1 gain	95% interval	McNemar p
Halfway: system vs flag every sprint	+0.0495	[0.006, 0.094]	< 0.001
Three-quarters: system vs flag every sprint	+0.0993	[0.049, 0.148]	< 0.001
Sprint end: system vs flag every sprint	+0.0549	[0.029, 0.080]	< 0.001
Sprint end: system vs tuned model	+0.1187	[-0.025, 0.291]	0.783

Ranking quality (AUC) mid-sprint

How well each signal orders late sprints above on-time ones. 0.5 is chance.

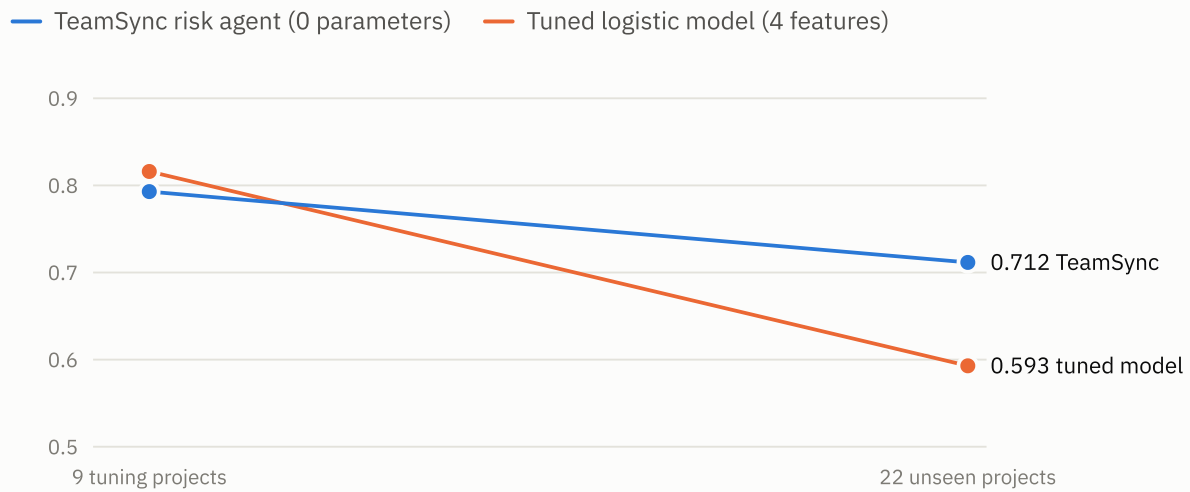
■ Projected unfinished share (TeamSync) ■ Share of work still open (simple count)



Checkpoint	TeamSync projection	Share of work still open
Halfway	0.7916	0.8018
Three-quarters	0.8744	0.8814

What happens across the project boundary

F1 at sprint end, on the 9 tuning projects and on the 22 unseen ones.



Predictor	Tuning projects	Unseen projects	Change
Tuned logistic model (cross-validated on tuning)	0.816	0.593	-0.223
TeamSync risk agent (no tuning)	0.793	0.712	-0.081
Tuned model AUC	0.887	0.657	-0.230

Where the errors fall – confusion matrices on held-out sprints

Each grid counts the 270 held-out sprints: rows are what really happened, columns what was predicted. Tinted blue = right, red = wrong.

Halfway

	Warned	Not warned
Late	119 caught	13 missed
On time	86 false alarm	52 correctly quiet

Right 63% · caught 90% of late · 38% of on-time left alone

Three-quarters

	Warned	Not warned
Late	127 caught	5 missed
On time	77 false alarm	61 correctly quiet

Right 70% · caught 96% of late · 44% of on-time left alone

Sprint end

	Warned	Not warned
Late	132 caught	0 missed
On time	107 false alarm	31 correctly quiet

Right 60% · caught 100% of late · 22% of on-time left alone

Sprint end, tuned model

	Warned	Not warned
Late	75 caught	57 missed
On time	46 false alarm	92 correctly quiet

Right 62% · caught 57% of late · 67% of on-time left alone

Calibration: the verdict's confidence value has a Brier score of 0.345 at sprint end on all 987 sprints (0 is perfect, 0.25 is a coin toss) — read the warnings as flags, not as probabilities. Sprint-end AUC is 0.581, weak by construction: every task in a sprint shares one deadline.

Predictor	Caught	Missed	False alarms	Correctly quiet	Precision	Recall	Specificity
Halfway	119	13	86	52	0.580	0.902	0.377
Three-quarters	127	5	77	61	0.623	0.962	0.442
Sprint end	132	0	107	31	0.552	1.000	0.225
Sprint end, tuned model	75	57	46	92	0.620	0.568	0.667

WHAT IT SHOWS

Halfway through a sprint the risk agent reaches **F1 0.706**, rising to **0.756** at three-quarters (0.803 counting only high-or-worse warnings). Against flagging every sprint the gains are **+0.050** and **+0.099**, and both intervals stay clear of zero ($p < 0.001$). It ranks late sprints well — AUC **0.792** and **0.874** — but no better than simply counting open work. The tuned model scores 0.816 on the projects it was tuned on and falls to 0.593 on unseen ones.

YOU CAN CLAIM

The system **warns about late sprints while they are still running**, significantly better than flagging everything, on projects it has never seen. Added model complexity **did not survive the project boundary**: the tuned model's advantage disappeared.

DON'T CLAIM

That the system *beats* the tuned model — at sprint end the two cannot be separated ($p = 0.78$, the interval crosses zero). That its projection ranks *better* than counting open work — it does not. That sprint-end recall of 1.00 is skill — after the deadline, the label and the rule test the same condition.

Do the specialist agents' warnings mean anything on their own?

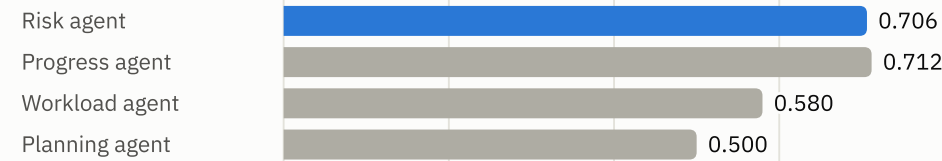
Planning, Progress and Workload each give a risk level of their own, shown to users beside the Risk agent's. They do not feed the product's risk score, but they are scored here against the same outcome: did the sprint finish late?

F1 of each agent's warning, held-out sprints

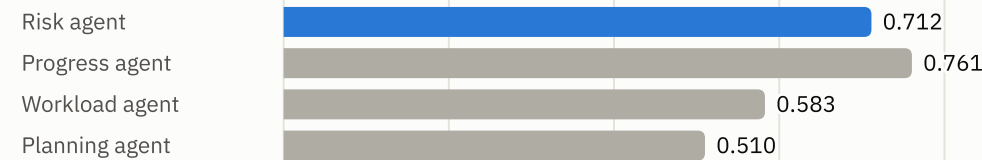
A warning is any risk level above low. 270 sprints from 22 unseen projects.

■ Risk agent — the product's verdict ■ Specialist agents

Halfway



Sprint end



Planning can only judge a sprint once capacity is known: 207 of 270 held-out sprints had a finished sprint before them. 77% of those planned more than the team's recent average, which is why it warns so often. Ranking quality of the plan-to-capacity ratio (AUC): 0.598 halfway, 0.616 at sprint end; the Risk agent's pace signal halfway: 0.792.

When	Agent	Warned	Precision	Recall	F1	Accuracy
Halfway	Risk agent	205 / 270	0.581	0.901	0.706	0.633
Halfway	Progress agent	208 / 270	0.582	0.917	0.712	0.637
Halfway	Workload agent	144 / 270	0.556	0.606	0.580	0.570
Halfway	Planning agent	160 / 270	0.456	0.553	0.500	0.459
Sprint end	Risk agent	239 / 270	0.552	1.000	0.712	0.604
Sprint end	Progress agent	81 / 270	1.000	0.614	0.761	0.811
Sprint end	Workload agent	146 / 270	0.555	0.614	0.583	0.570
Sprint end	Planning agent	166 / 270	0.458	0.576	0.510	0.459

WHAT IT SHOWS

The **Planning** agent now warns when a sprint plans more story points than the team can finish — the capacity set in project settings, or else the average its last three finished sprints completed. (Before this, no capacity ever reached it and it never warned.) Its warning is **weak**: F1 **0.500** halfway, below flagging every sprint (0.657), because most teams here plan more than they finish; the plan-to-capacity ratio ranks late sprints above on-time ones with AUC **0.60**, against 0.79 for the Risk agent. The **Progress** agent's simple completion rule matches the Risk agent halfway through (F1 0.712 vs 0.706); at sprint end its perfect precision is built in, since a sprint under half done is late by definition. The **Workload** agent is a weak lateness signal (F1 0.58) — it measures uneven load, not delay.

YOU CAN CLAIM

The product's verdict rests on the Risk agent, which is the one validated here; the specialists supply **descriptive context** whose figures are exactly right (section 1), the Planning agent's capacity and plan included.

DON'T CLAIM

That the Planning agent predicts late sprints: it is a planning check, and on this data a weak one. Its threshold was fixed in advance, not tuned on these sprints. Nor that the pace projection beats a plain completion-rate rule halfway through: they perform the same.

3 · SUGGESTED FIXES

Do the reassignments it suggests actually work?

For every real sprint with a workload recommendation, each suggested move was applied to the sprint and the risk recomputed, then compared with what the recommendation predicted before anyone acted.

Predicted effect held

1,524 / 1,524

new heaviest load and new severity, both exact

Severity dropped a level

62%

943 of 1,524 sprints

Heaviest load reduced by

6 points

median; mean 9.49

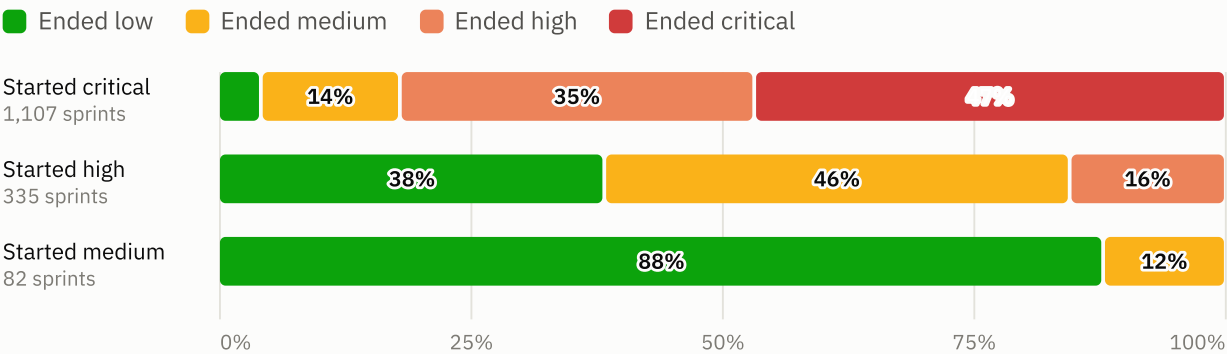
No helpful move existed

284

of 1,808 workload recommendations — none suggested

Where workload risk ended after the suggested moves

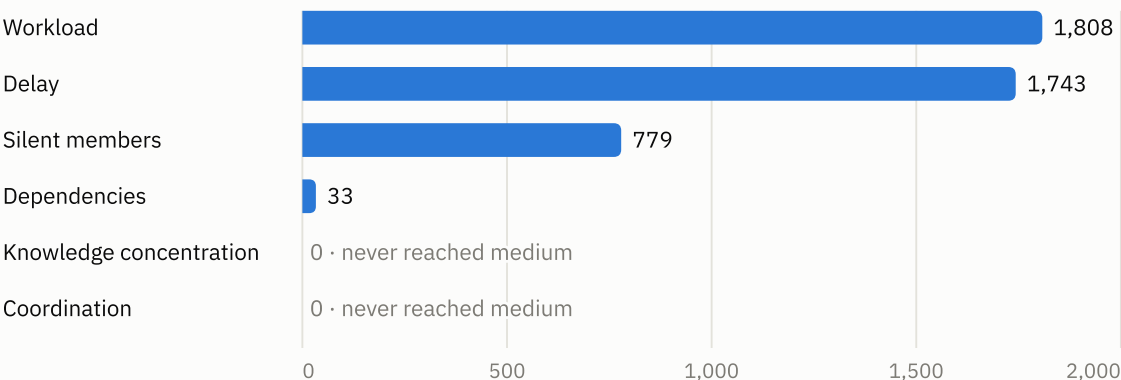
1,524 sprints, grouped by the severity they started at. The planner makes at most three moves.



Started	Ended low	Ended medium	Ended high	Ended critical
critical	45	153	390	519
high	128	155	52	0
medium	72	10	0	0

Recommendations made, by risk type

Every medium-or-worse risk across the 1,974 real-sprint analyses got one: 4,363 of 4,363.



Risk type	Recommendations made (one per medium-or-worse case)
Workload	1,808
Delay	1,743
Silent members	779
Dependencies	33
Knowledge concentration	0
Coordination	0

WHAT IT SHOWS

The predicted outcome held in **every one of 1,524 cases** — both the new heaviest load and the new severity. Severity dropped at least one level in **62%**; the heaviest load fell by a median of **6 story points**. Most of the rest started *critical*: 519 teams were too uneven for three moves to lift. In 284 sprints no move could help, and the recommendation says so rather than suggesting one.

YOU CAN CLAIM

The fixes are **actionable and honest**: each one names concrete moves, and its predicted effect is exactly what applying them produces.

DON'T CLAIM

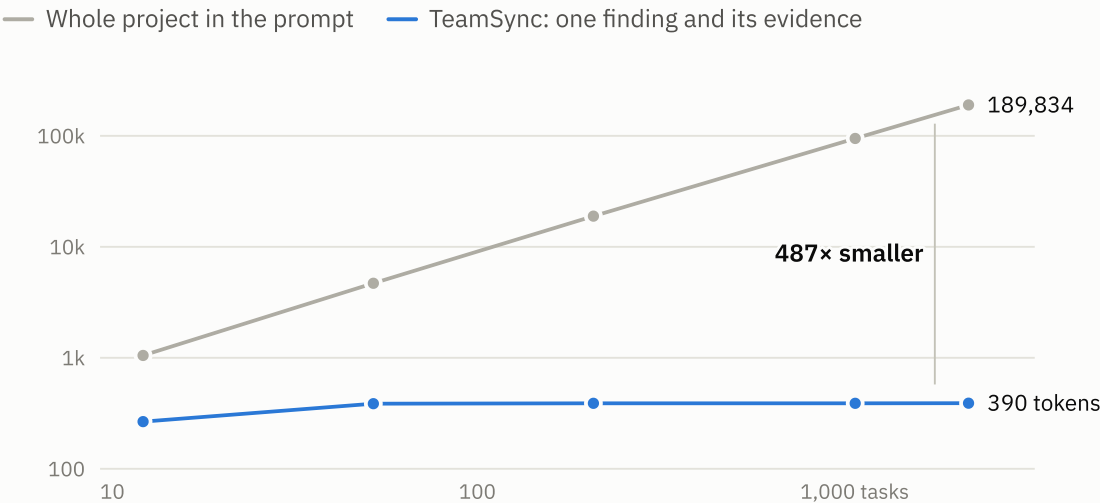
That the fixes improve real projects — this is measured inside the model, not with teams who followed them. And knowledge and coordination risk never reached *medium* on a real sprint, so their recommendations are exercised on generated data only.

4 • COST AND SPEED

Is it cheap and fast enough to run on every change?

Prompt size as the project grows

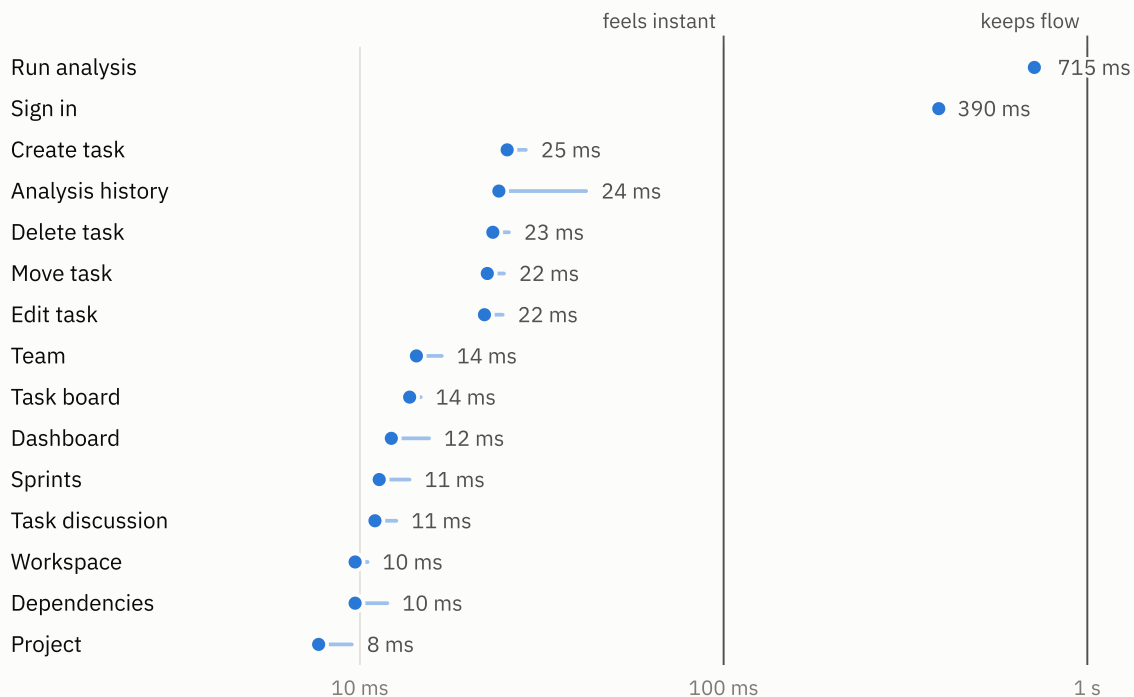
Tokens a language model would be sent. Both axes are logarithmic.



Tasks	Whole project (tokens)	TeamSync (tokens)	Ratio
13	1,050	266	4×
53	4,695	386	12×
203	18,901	389	49×
1,003	94,724	389	244×
2,003	189,834	390	487×
2,003 (re-measured)	200,916	390	515×

Response time of each feature, end to end

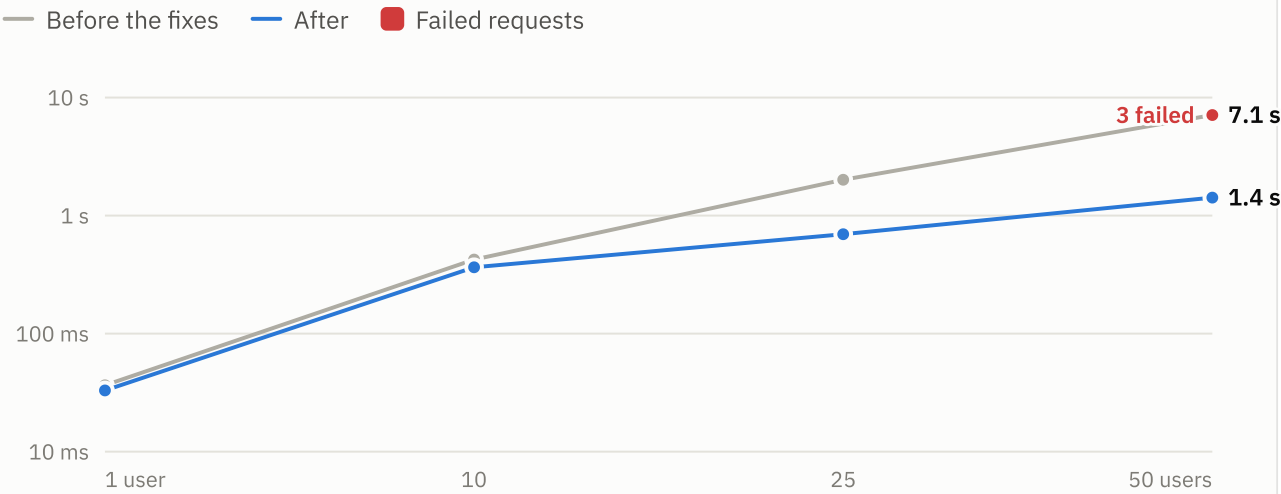
Real Mesos sprint (30 tasks, 13 people) over HTTP. Dot = median, line = up to the 95th percentile.
Logarithmic axis.



Feature	Median (ms)	95th percentile (ms)	Samples
Run analysis	715.1	753.9	10
Sign in	390.4	400.0	15
Create task	25.4	28.6	15
Analysis history	24.1	41.9	15
Delete task	23.2	25.7	15
Move task	22.4	24.9	15
Edit task	22.0	24.7	15
Team	14.3	16.8	15
Task board	13.7	14.7	15
Dashboard	12.2	15.5	15
Sprints	11.3	13.7	15
Task discussion	11.0	12.6	15
Workspace	9.7	10.5	15
Dependencies	9.7	11.9	15
Project	7.7	9.5	15

Many people at once: the slowest 1% of requests

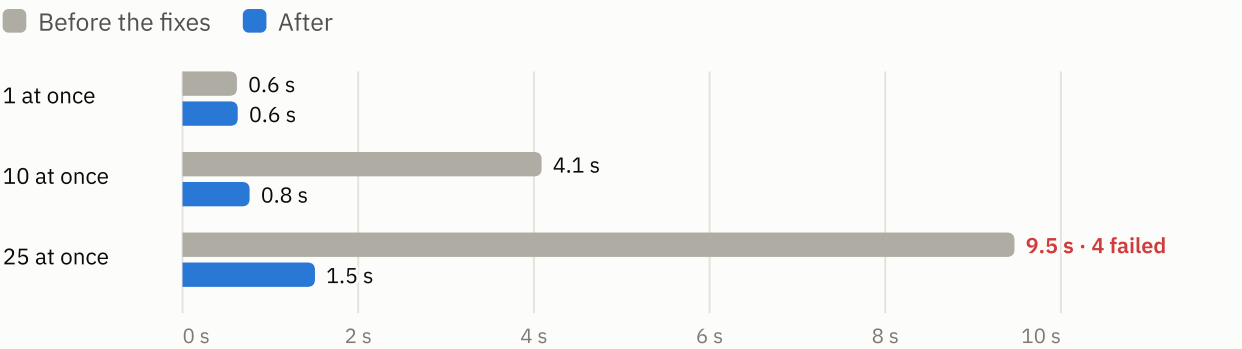
Simultaneous users repeating a session mix (board, dashboard, team, sprints, a discussion, create-move-delete a task) with no pause. Logarithmic axis.



Simultaneous users	Requests a second	Median	95th percentile	Slowest 1%	Failed requests
1	57.3 → 57.2	15 ms → 15 ms	31 ms → 27 ms	37 ms → 33 ms	0 → 0
10	59.4 → 76.8	148 ms → 122 ms	345 ms → 212 ms	424 ms → 365 ms	0 → 0
25	69.8 → 74	263 ms → 307 ms	746 ms → 603 ms	2.0 s → 695 ms	0 → 0
50	60.4 → 77.6	455 ms → 564 ms	2.4 s → 1.1 s	7.1 s → 1.4 s	3 → 0
1 analysis at once	—	719 ms → 732 ms	743 ms → 802 ms	—	0 → 0
5 analyses at once	—	1.9 s → 2.2 s	3.2 s → 3.6 s	—	0 → 0

A burst of sign-ins: the longest wait

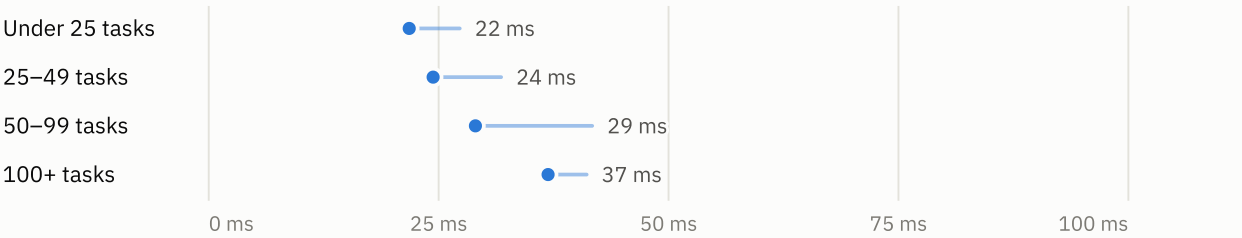
That many people signing in at the same moment. Password hashing is deliberately slow; it used to block every other request while it ran.



Signing in at once	Median wait	Longest wait	Failed
1	619 ms → 628 ms	619 ms → 628 ms	0 → 0
10	3.9 s → 719 ms	4.1 s → 764 ms	0 → 0
25	9.3 s → 1.1 s	9.5 s → 1.5 s	4 → 0

Analysis time by sprint size

The agent pipeline alone, in process, across 1,974 real-sprint analyses.



Sprint size	Analyses	Median (ms)	95th percentile (ms)
<25 tasks	1,129	21.8	27.3
25-49	726	24.4	31.8
50-99	115	29	41.7
100+	4	36.9	41.1

WHAT IT SHOWS

As the project grows from 13 to 2,003 tasks, the prompt grows only from 266 to **390 tokens (1.47×)**, while sending the whole project grows to 189,834 — **487× larger** (515× re-measured today, as tasks gained two fields). Every page and edit answers in **8–42 ms**; a full analysis, backend and AI service together, takes **715 ms**, of which the agent pipeline takes under 50 ms (22–37 ms median in the latest run, by sprint size). Sign-in's 390 ms is deliberate: password hashing is made slow to resist guessing. **Under load**, with up to 50 people at once and no pause between requests, the server peaks near **78 requests a second** with **no failed requests** and the slowest 1% within 1.4 s — after fixing three defects the load test itself found (7 failed requests before; 25 simultaneous sign-ins took 9.5 s).

YOU CAN CLAIM

Prompt cost **tracks the number of problems, not project size**, and the product is fast enough to re-analyse on every change: everything but a full analysis responds well inside the 100 ms that feels instant, and 50 simultaneous users cause waits, not errors.

DON'T CLAIM

Production-scale performance. These times and the load test are from one laptop on SQLite with a 30-task project, the load generator on the same machine; CI repeats the load test on PostgreSQL on a shared runner. Token counts are estimates from character length, not a tokenizer.

5 · THE OPTIONAL LANGUAGE MODEL

Is the language model safe to put in front of users?

The model is optional and only puts findings into words. These results come from the published runs on a rate-limited free key; re-measuring them needs a paid key.

Fabricated citations removed

200 / 200

before any reached a user

Agreement with the deterministic verdict

κ 0.95

Cohen's kappa, 37 comparisons — "almost perfect"

Accuracy vs. sending the whole project

1.000 vs 0.947

F1 on 36 identical scenarios; difference not significant

Calls made without a key

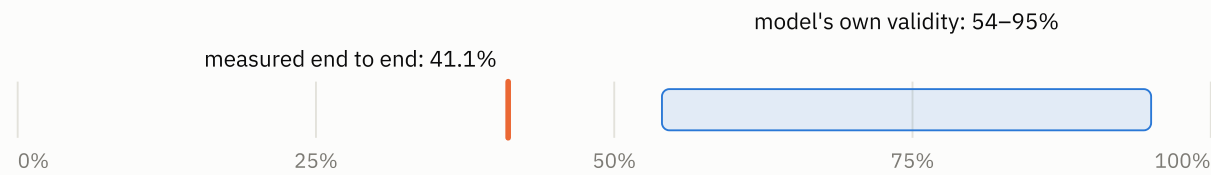
0

of 8,883 attempts reached the network

How often the model's output was valid

Only the end-to-end rate is measured. The model's own rate is bounded, because only 24 of 50 failures were diagnosed individually.

Measured end to end (rate limits included) Model's own validity: possible range



Risk type	Comparisons	Cohen's κ
coordination	1	1.00
delay	1	1.00
dependency	14	1.00
knowledge	12	0.86
silent member	1	1.00
workload	8	1.00
All	37	0.953
Validity end to end	90 calls	41.1%
Validity, model only	bounded	54-95%

WHAT IT SHOWS

Every fabricated citation injected in testing — **200 of 200** — was removed before reaching the user. The model's verdict agrees with the deterministic one almost perfectly (κ **0.95**). Only **41.1%** of calls produced valid output end to end, mostly because of free-tier rate limits. Without a key the model is never called: **0 of 8,883** attempts reached the network.

YOU CAN CLAIM

The model **narrates, it does not re-decide**, and it cannot put invented evidence in front of a user. The product loses nothing without it.

DON'T CLAIM

A validity rate for the model itself — it is a range, 54-95%. The κ of 0.95 rests on 37 comparisons, with three risk types at a single one; treat it as a first signal, not a settled number.

Is the software itself tested and safe?

Automated tests passing

250

backend 102 · AI service 85 · frontend 63

Role checks

185 / 185

every protected endpoint × every role

MVP code covered by tests

74% · 81%

backend MVP code · AI service (frontend, all code:
24%)

Defects found and fixed

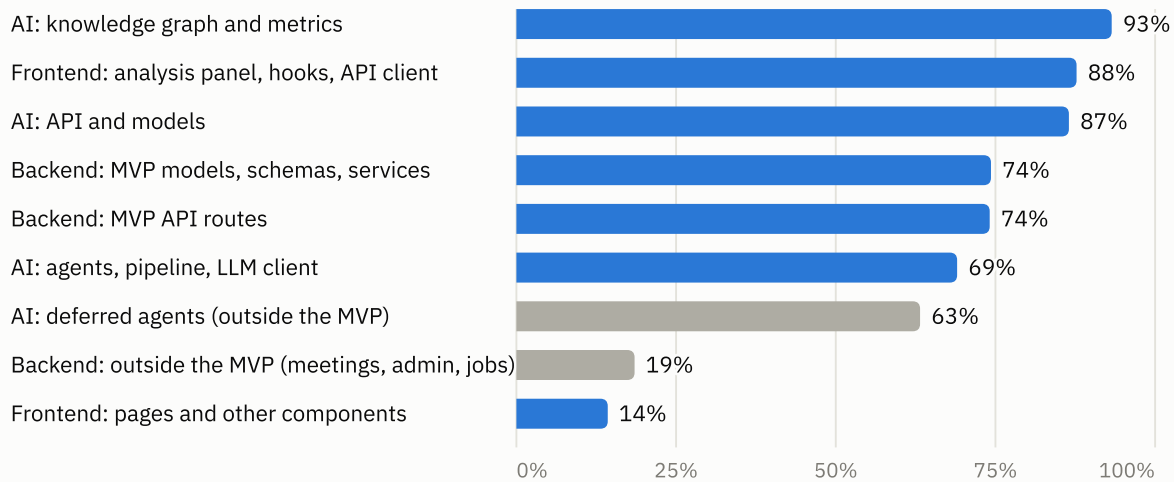
10

in this validation; 7 CI jobs re-check everything on
each push

Code covered by the automated tests, by area

Share of statements (for the frontend, lines) the test suites execute, every source file counted.

■ Part of the MVP ■ Outside the MVP (unused by the app)



Area	Statements covered
AI: knowledge graph and metrics	93.2%
Frontend: analysis panel, hooks, API client	87.7%
AI: API and models	86.5%
Backend: MVP models, schemas, services	74.3%
Backend: MVP API routes	74.1%
AI: agents, pipeline, LLM client	69.0%
AI: deferred agents (outside the MVP)	63.2%
Backend: outside the MVP (meetings, admin, jobs)	18.5%
Frontend: pages and other components	14.3%
Backend, all non-test code	60.1%
Backend, MVP code	74.2%
AI service, all non-test code	80.8%
Frontend, all source lines	24.0%

WHAT IT SHOWS

250 automated tests pass, the backend's on both SQLite and PostgreSQL, and a role matrix read off the code checks every protected endpoint against every role: **185 of 185**. Tests cover **74%** of the backend code the MVP uses, **81%** of the AI service and **93%** of the knowledge graph; in the frontend, **88%** of the analysis panel, hooks and API client, but only **24%** of all its code. The validation found and fixed ten defects: six security holes or crashes in endpoints the app does not use, a broken evaluation clock, and three that only showed under load.

YOU CAN CLAIM	The MVP is well tested where it matters , its access control is verified exhaustively rather than by example, and every push re-runs all of it.
DON'T CLAIM	High coverage overall: code outside the MVP is only 18% covered, and most frontend pages have no unit tests; they are exercised by the demo replay and by hand.

7 · IS THIS EVALUATION COMPLETE?

What is measured for each agent and feature — and what is not

A complete evaluation asks seven questions of every part. This matrix shows which are answered.

Part	Correct output	Accurate on real data	Compared with a baseline	Statistically tested	Speed and cost	Repeatable	Human evaluation
Coordinator	✓ measured	n/a	n/a	n/a	✓ measured	✓ measured	n/a
Planning agent	✓ measured	✓ measured: weak	✓ measured	✗ missing	✓ measured	✓ measured	✗ missing
Progress agent	✓ measured	✓ measured	⦿ partial	✗ missing	✓ measured	✓ measured	✗ missing
Workload agent	✓ measured	✓ measured	⦿ partial	✗ missing	✓ measured	✓ measured	✗ missing
Risk agent — delay	✓ measured	✓ measured	✓ measured	✓ measured	✓ measured	✓ measured	✗ missing
Risk agent — other five risks	✓ measured	⦿ partial	✗ missing	✗ missing	✓ measured	✓ measured	✗ missing
Recommendation agent	✓ measured	⦿ partial	✗ missing	n/a	✓ measured	✓ measured	✗ missing
Language-model narration (optional)	✓ measured	⦿ partial	✓ measured	⦿ partial	✓ measured	✗ missing	✗ missing
Web app and API	✓ measured	✓ measured	n/a	n/a	✓ measured, incl. load	✓ measured	✗ missing

Missing, in order of importance

Human evaluation of the explanations and the app. Does a manager find the right problem faster with the evidence than without? A usability score (SUS) for the app. The largest gap for an explainable-AI project, and no automated test can close it. Everything else is ready in docs/user-study/: protocol, forms, two real sprints, setup script, answer key, analysis script.	About 2 weeks · 8–12 participants
A better planning signal. The capacity warning now works but barely predicts lateness (AUC 0.60): most teams here plan more than they finish. Worth testing a plan measured against the team's own usual over-commitment.	Phase 2 · a few days
Load on production hardware. Load tested on one laptop (SQLite) and on a shared CI runner (PostgreSQL), up to 50 users. Not on a production server, nor beyond 50.	Half a day · a staging server
The other five risk types on real data. Knowledge and coordination never reach medium inside one sprint of TAWOS. Evaluate over releases, or on a dataset with component ownership.	Days to weeks · needs data
The language model, re-measured. Replace the 54–95% validity range with one number and thicken the k sample, with per-call logging.	About an hour · paid API key
Frontend page tests. Coverage is now measured: 24% of all frontend code. The analysis panel is well covered; most pages have no unit tests.	1–2 days

Limits of the data behind these numbers

- **The delay label is a proxy.** TAWOS has no due dates, so "late" means 30% or more of a sprint's issues unresolved at its end.
- **The first quarter of a sprint.** The pace warning is off by design until a quarter of the schedule has passed.
- **One machine.** Response times and the load test come from one laptop, on SQLite, with a 30-task project; CI repeats the load test on PostgreSQL.
- **The language-model results are from earlier runs** on a rate-limited free key; they were not re-measured.

Every figure on this page is read from committed result files: `ai-service/eval/phase1_validation_result.json`, `phase1_api_latency_result.json`, `phase1_load_test_result.json`, `datasets/tawos_midsprint_result.json`, `datasets/tawos_holdout_result.json`, `figure_token_sweep.json`, `live_llm_result_v3.json`.

Reproduce: `python -m eval.validate_phase1` (from `ai-service/`) and `python -m app.scripts.measure_latency / load_test --quick` (from `backend/`).